

Mata Kuliah : **Praktikum Aplikasi Berbasis Platform**
Pertemuan : **9 – 10**
Nama : **Naufal Thoriq Muzhaffar**
NIM : **2311102078**

1. Deskripsi Aplikasi

Task Tracker adalah aplikasi manajemen tugas berbasis Flutter yang dikembangkan pada Praktikum Pertemuan 9–10. Aplikasi ini mendemonstrasikan dua konsep utama: (1) pengelolaan state reaktif menggunakan library Provider berbasis pola ChangeNotifier, dan (2) integrasi layanan push notification menggunakan Firebase Cloud Messaging (FCM).

Seluruh logika state terpusat pada class TaskProvider yang menyimpan daftar tugas sebagai List<Map>. Perubahan pada state—baik penambahan maupun penghapusan tugas—dipropagasi secara otomatis ke seluruh widget yang menggunakan Consumer<TaskProvider> tanpa perlu melakukan setState secara manual.

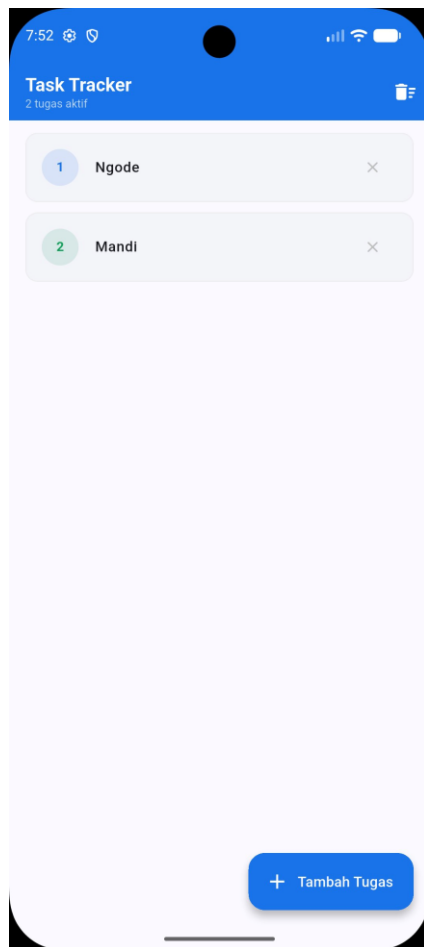
Tabel Fitur Implementasi

Fitur	Teknologi	Keterangan
State Management	Provider + ChangeNotifier	Reaktif, tanpa setState manual
Tambah Tugas	AlertDialog + TextField	Via FloatingActionButton
Hapus Satu	IconButton per card	removeTask(id) pada Provider
Hapus Semua	AppBar TrashIcon	clearAll() + konfirmasi dialog
FCM Foreground	onMessage.listen	SnackBar biru muncul di layar
FCM Background	Top-level handler	Notifikasi tray sistem Android
FCM Token	getToken()	Dicetak di debug console

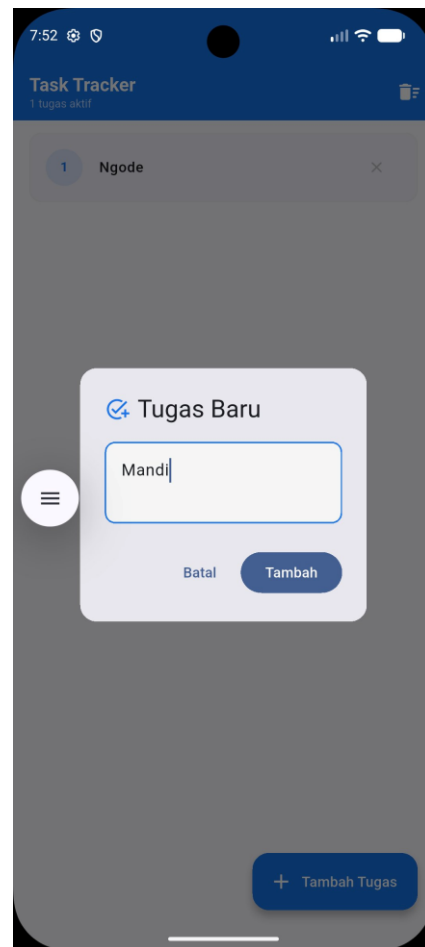
2. Hasil Implementasi

2.1 Tampilan Daftar Tugas & Penambahan Tugas Baru

Gambar 1 menampilkan state home screen setelah dummy data awal dihapus dan pengguna menambahkan dua tugas baru: "Nkode" dan "Mandi". Counter "2 tugas aktif" di bawah judul AppBar diperbarui secara reaktif melalui Consumer<TaskProvider> setiap kali daftar berubah. Gambar 2 menunjukkan dialog input yang muncul ketika FloatingActionButton ditekan, lengkap dengan TextField autofocus dan tombol aksi "Batal" / "Tambah".



Gambar 1. Tampilan daftar tugas aktif

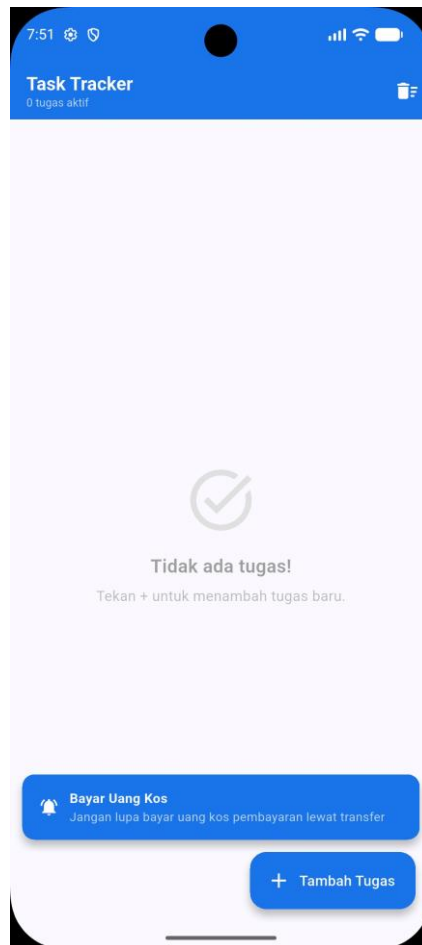


Gambar 2. Dialog penambahan tugas baru

Arsitektur Provider: Widget Consumer<TaskProvider> yang membungkus ListView.builder akan otomatis rebuild ketika TaskProvider memanggil notifyListeners(). Pendekatan ini memisahkan UI dari logika bisnis sehingga kode lebih terstruktur dan mudah diuji (separation of concerns).

2.2 Penerimaan Notifikasi FCM (Foreground)

Pengujian notifikasi dilakukan melalui Firebase Console dengan mengirimkan pesan ke FCM Token perangkat yang diperoleh dari debug console saat aplikasi pertama kali dijalankan. Notifikasi dengan judul "Bayar Uang Kos" dan isi "Jangan lupa bayar uang kos pembayaran lewat transfer" berhasil diterima oleh listener FirebaseMessaging.onMessage.listen yang aktif di initState().

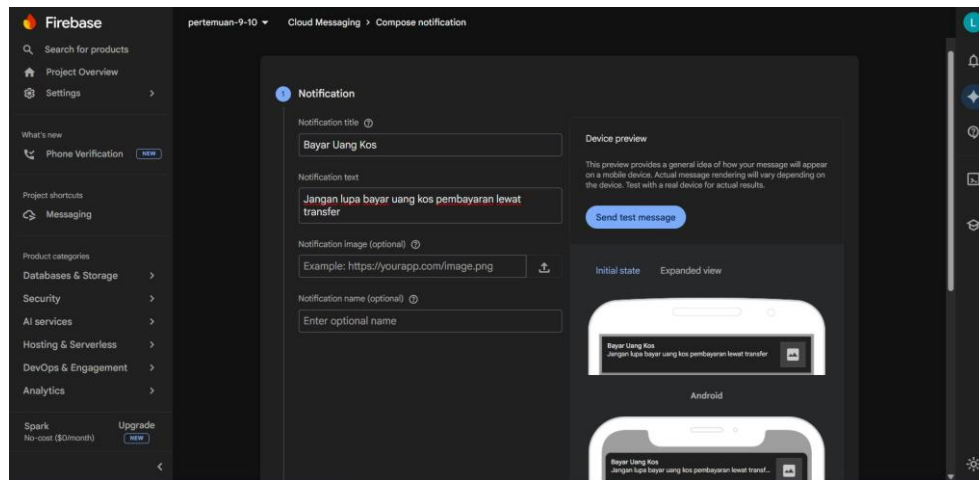


Gambar 3. Snackbar biru muncul saat notifikasi FCM diterima dalam kondisi aplikasi terbuka (foreground)

Catatan teknis: Snackbar ditampilkan dengan properti behavior: `SnackbarBehavior.floating`, memuat ikon lonceng, judul tebal, dan teks body—seluruhnya diambil dari objek `RemoteMessage.notification` yang dikirim oleh FCM server.

2.3 Konfigurasi Pengiriman Notifikasi via Firebase Console

Gambar 4 memperlihatkan tampilan Firebase Console—halaman Compose Notification pada project "pertemuan-9-10". Kolom Notification title diisi "Bayar Uang Kos" dan Notification text diisi pesan pengingat tagihan. Device preview di sisi kanan menampilkan pratinjau tampilan notifikasi pada perangkat Android dan iOS sebelum dikirim.



Gambar 4. Tampilan Firebase Console — Compose Notification pada project pertemuan-9-10

3. Analisis dan Kesimpulan

Implementasi State Management dengan Provider terbukti memberikan alur data yang bersih dan terpisah antara layer UI dan layer logika. Penggunaan ChangeNotifier memungkinkan komponen yang tidak saling berhubungan secara langsung tetap dapat sinkron terhadap perubahan state yang sama, seperti counter jumlah tugas di AppBar dan daftar card di body layar.

Integrasi Firebase Cloud Messaging berjalan sebagaimana mestinya di ketiga skenario: foreground (SnackBar), background (notifikasi tray), dan terminated (notifikasi tray). Konfigurasi menggunakan FlutterFire CLI mempercepat setup karena secara otomatis menghasilkan file `firebase_options.dart` dan memodifikasi `build.gradle` tanpa perlu konfigurasi manual yang rawan kesalahan.

Poin pembelajaran utama: background message handler wajib dideklarasikan sebagai top-level function (bukan method di dalam class) agar dapat dieksekusi oleh Flutter di Isolate terpisah yang dikelola oleh FCM SDK.